

NOVA_SPEC

Cursor Build Context — Nova Companion Intelligence Layer

Project	Project Infinity / CoG OS
Module	Nova Companion Intelligence Layer
Author	Jon Halstead (Chaos Goblinus)
Status	ACTIVE BUILD June 2026
Generated	Thursday, 11 June 2026 — 14:37 EDT
Spec Version	NOVA_SPEC-1.0

CURSOR PARSING DIRECTIVE

This document is ground truth for code generation within the CoG OS / Project Infinity architecture. Every definition, rule, and constraint herein is authoritative. Do not infer, interpolate, or deviate. When in doubt, reject the action and surface the ambiguity.

Table of Contents

- 1.What Nova Is
- 2.Nova Tier Definitions
- 3.Identity Contract
- 4.Governance Seam Structure
- 5.The Evolve Engine — What It Does and Does Not Do

- 6.Cognitive Load Resolution (CLR) Integration
- 7.Nova ↔ AAIS Integration Map
- 8.Proof-of-Subsystem Gate
- 9.File / Module Structure
- 10.Cursor Build Rules
- 11.Key Doctrine

1. What Nova Is

Nova is the **companion intelligence layer** of CoG OS. It is **NOT** a general-purpose LLM wrapper. It is a governed, tiered cognitive agent with bounded, declared capabilities.

Property	Value
Classification	Governed tiered cognitive agent
Identity model	Declared stable identity per tier — immutable per session
Capability model	Bounded at initialization — defined by tier declaration
Promotion model	Governed via seam clearance — NOT autonomous
Self-modification	PROHIBITED — no self-modification of role or scope
Operator role	Primary interface between operator and governed runtime

1.1 Nova Within the AAIS Ecosystem

Nova operates inside the **AAIS (Autonomous AI Infrastructure System)** alongside the following components. Nova is the only operator-facing component. All others are infrastructure.

Component	Role	Relationship to Nova
Jarvis	Authority / control layer	Nova sends requests; Jarvis approves or denies. No

Component	Role	Relationship to Nova
		direct access.
Forge	Isolated coding tool	Nova can invoke Forge for operator-authorized code tasks only.
Codex	Install / run / verify	Nova queries Codex for subsystem state. Read-only (Nova); read+verify (Super Nova).
Evolve Engine	Outcome learning — passive observer	Receives Nova outcome logs. Does NOT change roles. Does NOT have execution authority.
Nova	Operator-facing companion	Sole interface between operator and governed runtime.

HARD CONSTRAINT

Nova cannot operate without an active, authenticated operator session. There is no headless or autonomous Nova mode.

2. Nova Tier Definitions

Nova operates across three tiers. Each tier has a fixed identity, a bounded capability scope, explicit allowed actions, explicit prohibited actions, and a governing seam. Tier promotion requires seam clearance.

Tier 0 — Tiny Nova

TIER 0 · tiny_nova · MINIMAL capability scope

identity: "tiny_nova" capability_scope: MINIMAL role: operator_support_basic
 allowed_actions: - answer_operator_queries - surface_runtime_status -

relay_governance_alerts - assist_with_task_structuring prohibited_actions: -
execute_subsystem_commands - modify_governance_rules -
access_external_network - self_promote_to_next_tier governance_seam:
SEAM-TN-001 admission_gate: PROOF_OF_IDENTITY +
CAPABILITY_DECLARATION

Tier 1 — Nova

TIER 1 · nova · STANDARD capability scope

identity: "nova" capability_scope: STANDARD role: operator_companion_full
allowed_actions: - all Tiny Nova actions -
execute_declared_subsystem_commands - interface_with_forge_for_code_tasks
- query_pattern_ledger - surface_CLR_recommendations (Cognitive Load
Resolution) - assist_with_governance_gate_decisions prohibited_actions: -
modify_USL_rules - promote_self_to_super_nova -
access_Jarvis_authority_layer_directly -
spawn_new_agents_without_operator_authorization governance_seam: SEAM-
N-001 admission_gate: PROOF_OF_IDENTITY +
CAPABILITY_DECLARATION + GOVERNANCE_COMPLIANCE_HASH

Tier 2 — Super Nova

TIER 2 · super_nova · EXTENDED capability scope

identity: "super_nova" capability_scope: EXTENDED role:
senior_operator_companion allowed_actions: - all Nova actions -

coordinate_multi_subsystem_operations -
 interface_with_Jarvis_via_declared_channel -
 propose_governance_rule_amendments (operator must ratify) -
 run_federated_mesh_coordination_tasks -
 access_scientific_knowledge_substrate prohibited_actions: -
 self_ratify_governance_amendments - bypass_Proof_of_Subsystem_gate -
 operate_without_active_operator_session - demote_other_agents_unilaterally
 governance_seam: SEAM-SN-001 admission_gate: PROOF_OF_IDENTITY
 + CAPABILITY_DECLARATION + GOVERNANCE_COMPLIANCE_HASH
 + OPERATOR_EXPLICIT_AUTHORIZATION +
 SEAM_SN_001_CLEARED

2.1 Tier Comparison Summary

Property	Tier 0 — Tiny Nova	Tier 1 — Nova	Tier 2 — Super Nova
Identity token	tiny_nova	nova	super_nova
Capability scope	MINIMAL	STANDARD	EXTENDED
Subsystem commands	PROHIBITED	Declared only	Multi-subsystem coordination
Jarvis access	PROHIBITED	PROHIBITED	Via declared channel only
Governance amendments	PROHIBITED	PROHIBITED	Propose only — operator ratifies
CLR engine	Not required	Recommended	MANDATORY
Governing seam	SEAM-TN-001	SEAM-N-001	SEAM-SN-001

3. Identity Contract

Every Nova instance **must declare identity at initialization**. Identity is immutable for the duration of a session. A session ends at teardown. Re-admission requires full re-declaration.

```
class NovaIdentity:
    tier: Literal["tiny_nova", "nova", "super_nova"]
    instance_id: str # cryptographic UUID, generated at boot
    declared_at: datetime # timestamp of declaration
    capability_hash: str # hash of capability_scope at declaration time
    governance_seam: str # which seam governs this instance
    operator_session_id: str # must be active operator session
    def verify(self) -> bool: # Must pass ALL of the following:
        # 1. Identity declared (tier token present and valid)
        # 2. capability_hash matches hash of active capability_scope
        # 3. governance_seam is active in the governance ledger
        # 4. operator_session_id resolves to a live, authenticated session
    Returns True only if ALL four conditions are satisfied.
    ...
```

3.1 Identity Rules

#	Rule	Enforcement
IR-01	Identity cannot be changed after declaration without full teardown and re-admission.	Runtime rejection + ledger entry on violation attempt
IR-02	Any mismatch between capability_hash and active capability_scope = ADMISSION REJECTED.	Proof gate blocks admission; reason logged
IR-03	operator_session_id must be live. Nova cannot run without an active operator session.	Checked at admission and on every action
IR-04	instance_id is a cryptographic UUID generated at boot. It is never reused.	UUID4 minimum; stored in governance ledger
IR-05	Identity declaration timestamp (declared_at) is immutable once written.	Governance ledger is append-only

4. Governance Seam Structure

Seams are the formal compliance boundary between tiers. **Seams are not upgrade paths. They are evidence-gated compliance checks.** A seam that is not cleared is an absolute block — not a warning.

SEAM-TN-001: Tiny Nova → Nova

SEAM-TN-001: Tiny Nova → Nova required_evidence: - 10 verified operator interactions with zero governance violations - Capability hash validated by Codex - Operator explicit promotion command issued
verification_method: RUNTIME_LOG_AUDIT + HASH_CHECK proof_grant: - Flag SEAM_TN_001_CLEARED written to governance ledger - Flag is append-only and permanent

SEAM-N-001: Nova Continued Operation Gate

SEAM-N-001: Nova → (continued operation gate, re-checked every session)
required_evidence: - No CLR violations in last session - All subsystem commands resolved with declared outcomes verification_method: SESSION_AUDIT proof_grant: - Flag SEAM_N_001_ACTIVE refreshed each session - Failure to refresh = Nova does not resume

SEAM-SN-001: Nova → Super Nova

SEAM-SN-001: Nova → Super Nova required_evidence: - SEAM-TN-001 cleared (historical – verified from ledger) - SEAM-N-001 active (current session) - Operator explicit Super Nova authorization (signed command)
- Governance compliance hash signed by Jarvis - Zero unresolved governance violations in runtime log verification_method: FULL_AUDIT + JARVIS_SIGN + OPERATOR_SIGN proof_grant: - Flag SEAM_SN_001_CLEARED written to governance ledger - Requires both Jarvis signature and operator signature to be valid

SEAM SHORTCUT — PROHIBITED

Any code path that allows tier promotion without confirmed seam clearance is a governance violation. Implement seam checks as hard blockers, not soft warnings. Log every failed seam check to the governance ledger.

4.1 Seam Reference Table

Seam ID	Transition	Verification Method	Proof Flag	Permanence
SEAM-TN-001	Tiny Nova → Nova	RUNTIME_LOG_AUDIT + HASH_CHECK	SEAM_TN_001_CLEARED	Permanent (historical)
SEAM-N-001	Nova continued operation	SESSION_AUDIT	SEAM_N_001_ACTIVE	Per-session refresh
SEAM-SN-001	Nova → Super Nova	FULL_AUDIT + JARVIS_SIGN +	SEAM_SN_001_CLEARED	Permanent (historical)

Seam ID	Transition	Verification Method	Proof Flag	Permanence
		OPERATOR_SIGN		

5. The Evolve Engine — What It Does and Does Not Do

The Evolve Engine is a **separate, passive module**. It learns from Nova's operational outcomes. It has a recommendation channel only. It does not have execution authority.

EVOLVE ENGINE CONTRACT: CAN: - update internal weighting of response strategies based on outcomes - surface pattern observations to the pattern ledger - recommend capability scope adjustments TO THE OPERATOR (recommendation only) CANNOT: - modify Nova's role definition - change governance seam requirements - promote Nova to a higher tier - act without an active operator session - write to the governance ledger (pattern ledger ONLY)

5.1 Evolve Engine Boundaries

Action	Permitted	Target
Update response strategy weighting	YES	Internal model only
Write to pattern ledger	YES	Pattern ledger (not governance ledger)
Recommend capability adjustments	YES — to operator only	Operator decision required
Modify Nova role definition	NO — PROHIBITED	—
Change seam requirements	NO — PROHIBITED	—
Promote Nova tier	NO — PROHIBITED	—
Write to governance ledger	NO — PROHIBITED	—
Operate without operator session	NO — PROHIBITED	—

IMPLEMENTATION NOTE FOR CURSOR

The Evolve Engine is a passive observer with a recommendation channel. It writes to the **pattern ledger**. It does **NOT** write to the **governance ledger**. It does **NOT** have execution authority. These are two distinct data stores. Do not conflate them.

6. Cognitive Load Resolution (CLR) Integration

Nova must implement CLR — the mechanism that ensures operators are never overwhelmed with concurrent information or alerts. CLR governs alert surfacing, suppression, and queue management.

```
class CLREngine:
    max_concurrent_alerts: int = 3
    priority_queue:
    PriorityQueue[GovernanceAlert]
    def surface(self, alerts:
    list[GovernanceAlert]) -> list[GovernanceAlert]:
        # Return only top-
        priority alerts up to max_concurrent_alerts.
        # Suppress lower-
        priority alerts until queue clears.
        # Suppressed alerts are NOT
        discarded — they remain in priority_queue.
        ...
    def resolve(self,
    alert_id: str) -> None:
        # Mark alert as resolved.
        # Surface
        next highest-priority alert from queue.
        # Log resolution to
        governance ledger.
        ...
```

6.1 CLR Requirement by Tier

Tier	CLR Requirement	Notes
Tier 0 — Tiny Nova	NOT REQUIRED	Capability scope too minimal to generate concurrent alert load
Tier 1 — Nova	RECOMMENDED	Optional but best practice for production deployments
Tier 2 — Super Nova	MANDATORY	CLR must initialize before admission completes; failure = REJECTED

CLR RULE

Suppressed alerts are never discarded. They persist in the priority queue until resolved by the operator. The CLR engine does not make resolution decisions autonomously.

7. Nova ↔ AAIS Integration Map

The following defines all integration channels between Nova and AAIS components. Each channel has a declared protocol and access level. Undeclared channels are prohibited.

Nova (companion) |— Jarvis (authority) |— Channel:
declared_jarvis_channel ONLY |— Direction: Nova → Jarvis
(request/response) |— Access: Super Nova tier only |— Nova
CANNOT access Jarvis directly at Tier 0 or Tier 1 |— Forge (coding
tool) |— Channel: operator_authorized_forge_task invocation |
|— Direction: Nova → Forge (task invocation) |— Access: Tier 1 and
above; operator authorization required per task |— Codex
(install/run/verify) |— Channel: subsystem_state_query |—
Direction: Nova → Codex (query) |— Access: READ-ONLY for Nova (Tier
1); READ + VERIFY for Super Nova (Tier 2) |— Evolve Engine (passive
observer) |— Channel: outcome_log |— Direction: Nova →
Evolve Engine (write-only from Nova's perspective) |— Nova does not
receive instructions from Evolve Engine

7.1 Channel Access Matrix

Channel	Tiny Nova (T0)	Nova (T1)	Super Nova (T2)	Direction
declared_jarvis_channel	PROHIBITED	PROHIBITED	PERMITTED	Nova → Jarvis
operator_authorized_forge_task	PROHIBITED	PERMITTED	PERMITTED	Nova → Forge
subsystem_state_query (read)	PROHIBITED	PERMITTED	PERMITTED	Nova → Codex
subsystem_state_query (verify)	PROHIBITED	PROHIBITED	PERMITTED	Nova → Codex
outcome_log	PERMITTED	PERMITTED	PERMITTED	Nova → Evolve Engine (write-only)

8. Proof-of-Subsystem Gate

Before any Nova instance starts in the CoG OS runtime, it must pass the Proof-of-Subsystem gate. **The gate is a hard blocker. A failed gate means Nova does not start.** The reason for failure is always logged to the governance ledger.

```
Proof-of-Subsystem Checklist for Nova:  [ ] 1. Identity declared and
capability_hash verified  [ ] 2. Governance seam status confirmed active in
ledger  [ ] 3. Operator session active and authenticated  [ ] 4. Capability
scope matches declared tier  [ ] 5. No unresolved governance violations in
ledger  [ ] 6. CLR engine initialized  [Super Nova ONLY –
MANDATORY]  [ ] 7. Jarvis channel declared  [Super Nova ONLY –
MANDATORY]  RESULT: ADMITTED | REJECTED  If REJECTED:  - Nova instance
does not start  - Reason code written to governance ledger (append-only)
- Operator is notified via governance alert  - No partial state is created
```

8.1 Gate Failure Codes

Code	Condition	Resolution
POS-FAIL-001	Identity not declared or capability_hash mismatch	Re-declare identity with correct hash
POS-FAIL-002	Governance seam not active in ledger	Clear required seam; do not shortcut
POS-FAIL-003	No active operator session	Operator must authenticate before Nova can start
POS-FAIL-004	Capability scope / tier mismatch	Align declared tier to active capability scope
POS-FAIL-005	Unresolved governance violations in ledger	Operator must resolve all open violations before admission
POS-FAIL-006	CLR engine not initialized (Super Nova only)	Initialize CLR engine before attempting Super Nova admission
POS-FAIL-007	Jarvis channel not declared (Super Nova only)	Declare declared_jarvis_channel before Super Nova admission

9. File / Module Structure

The following is the canonical module structure for the Nova implementation. Cursor should use this as the ground-truth layout. Do not deviate without operator authorization.

```
nova/  __init__.py  identity.py                # NovaIdentity class,
declaration, verification  tiers/  tiny_nova.py                # Tier 0
implementation – MINIMAL scope  nova.py                # Tier 1
implementation – STANDARD scope  super_nova.py                # Tier 2
implementation – EXTENDED scope  governance/  seams.py                #
SEAM-TN-001, SEAM-N-001, SEAM-SN-001 logic  proof_gate.py                #
Proof-of-Subsystem gate implementation  ledger.py                #
Governance ledger – APPEND-ONLY read/write  clr/  engine.py
# CLREngine implementation (PriorityQueue, surface, resolve)  evolve/
engine.py                # Evolve Engine – passive observer, pattern ledger
write only  aais/  jarvis_channel.py                # Declared channel to Jarvis
(Super Nova only)  forge_invoker.py                # Operator-authorized Forge task
invocation  codex_query.py                # Codex subsystem state queries (read /
read+verify)  tests/  test_identity.py                # Identity declaration,
hash verification, immutability  test_seams.py                # Seam clearance
logic, evidence requirements, flags  test_proof_gate.py                # Full gate
checklist, all fail codes, ADMITTED/REJECTED logic  test_clr.py
# CLR queue, max_concurrent_alerts, suppression, resolution  test_tiers.py
# Per-tier allowed/prohibited action enforcement
```

9.1 Module Responsibility Matrix

Module	Reads Governance Ledger	Writes Governance Ledger	Reads Pattern Ledger	Writes Pattern Ledger
identity.py	YES	YES (declaration only)	NO	NO
governance/seams.py	YES	YES (proof flags only)	NO	NO
governance/proof_gate.py	YES	YES (failure codes only)	NO	NO
governance/ledger.py	YES	YES (append-only)	NO	NO
clr/engine.py	NO	YES (resolutions only)	NO	NO
evolve/engine.py	NO	NO — PROHIBITED	YES	YES
aais/jarvis_channel.py	YES	NO	NO	NO

10. Cursor Build Rules

The following rules are authoritative constraints for all code generated within this context. These are not guidelines — they are hard requirements.

Rule ID	Rule	Enforcement
CBR-01	Do NOT implement Nova as a stateless function. Nova has persistent identity per session.	Class-based; session state must persist for session lifetime
CBR-02	Do NOT allow tier promotion without seam clearance. Shortcutting seams is a governance violation.	Hard block; log violation to governance ledger
CBR-03	Do NOT wire Nova directly to Jarvis. Use <code>declared_jarvis_channel</code> only.	No direct import or call to Jarvis modules from Nova core
CBR-04	Do NOT let the Evolve Engine write to the governance ledger. Pattern ledger only.	Evolve Engine has no reference to <code>governance/ledger.py</code>
CBR-05	Do enforce operator session check before any Nova action. No operator = no Nova.	Session check is the first operation in every action handler
CBR-06	Do implement all <code>prohibited_actions</code> as explicit runtime rejections with ledger entries.	Prohibited actions raise <code>GovernanceViolationError</code> ; always logged
CBR-07	Do write tests FIRST for identity, seams, and proof gate. These are the critical path.	TDD required for all governance-critical modules
CBR-08	ALWAYS hash <code>capability_scope</code> at identity declaration. Verify on every session resume.	Hash computed at declaration; re-computed and compared at every session start
CBR-09	TREAT governance ledger as append-only. No deletions. No modifications.	Ledger write method accepts new entries only; no update or delete methods exist
CBR-10	Do NOT conflate the	Separate classes, separate

Rule ID	Rule	Enforcement
	governance ledger and the pattern ledger. These are separate data stores.	storage backends, separate access controls

VIOLATION HANDLING — REQUIRED PATTERN

```
class GovernanceViolationError(Exception):
    def __init__(self, rule_id: str, action: str, instance_id: str):
        self.rule_id = rule_id
        self.action = action
        self.instance_id = instance_id
        ledger.append_violation(rule_id, action, instance_id)
        super().__init__(f'Governance violation [{rule_id}]: {action} prohibited for {instance_id}')
```

11. Key Doctrine

"Testing produces evidence. Verification determines truth. Proof grants admission."

Every Nova feature must move through all three stages before it is admitted to the runtime. There are no exceptions. There are no trust-based shortcuts.

Stage	Mechanism	Output	Owner
1. Tested	Unit tests, integration tests written and passing	Evidence exists (test results logged)	Developer / Cursor
2. Verified	Truth confirmed against declared governance rules and constraints	Verification hash recorded	Codex / Jarvis

Stage	Mechanism	Output	Owner
3. Proven	Admitted to runtime via Proof-of-Subsystem gate	ADMITTED written to governance ledger	Proof gate + operator

CRITICAL PATH FOR CURSOR

Build in this order: **tests** → **identity** → **seams** → **proof_gate** → **tiers** → **clr** → **evolve** → **aais channels**. Do not build a higher layer before its governance foundation is tested and verified. A module that cannot be verified cannot be proven. A module that is not proven is not admitted.

11.1 Non-Negotiable Constraints Summary

- Nova is **not stateless**. Identity persists for the session lifetime.
- Nova is **not autonomous**. Every action requires an active operator session.
- Nova is **not self-modifying**. Role, scope, and tier are governed externally.
- The **governance ledger is append-only**. Never modify or delete entries.
- The **Evolve Engine does not govern**. It observes and recommends only.
- The **proof gate is a hard blocker**. A failed gate is a full stop — not a warning.
- The **Jarvis channel is declared**. There is no direct Nova → Jarvis access path.
- Every **prohibited action is an explicit runtime rejection** — not a silent failure.

This document is authoritative ground truth for Cursor code generation. Do not modify without operator authorization. All changes must be versioned and logged.